HW2 protocol

Edward Student

Git: https://github.com/EdvardStudent23/apz_hw/tree/hazelcast

I have installed hazelcast and configured nodes like here:

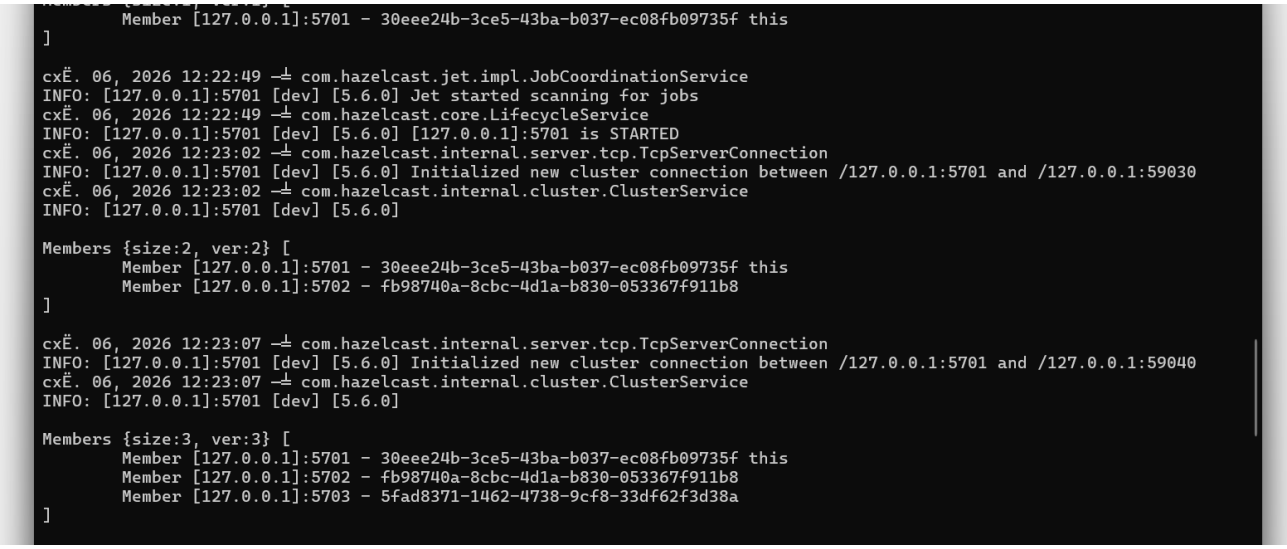
Встановити і налаштувати Hazelcast https://hazelcast.com/open-source-projects/downloads/

Сконфігурувати і запустити 3 ноди (інстанси) об'єднані в кластер або як частину Java-застосування, або як окремі застосування https://docs.hazelcast.com/hazelcast/5.3/getting-started/get-started-binary#step-6-scale-your-cluster

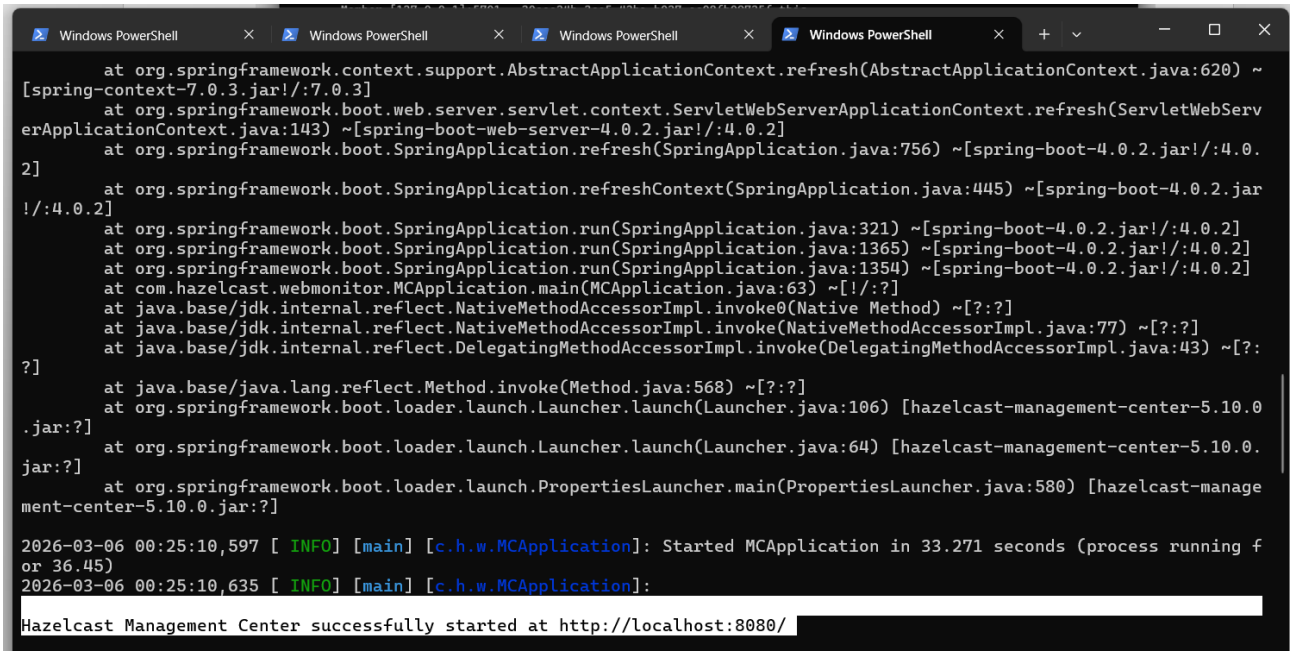
Task 1-2:

I have started 3 hazelcast nodes using binary version which I installed:

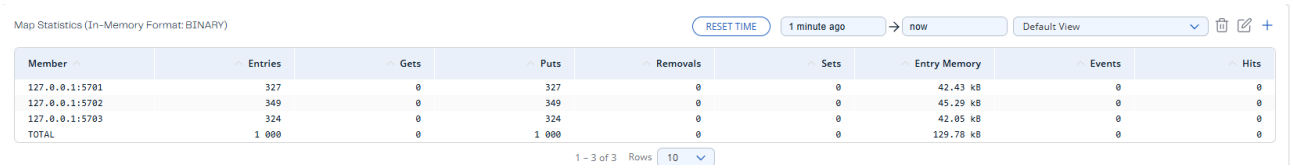
3 nodes:



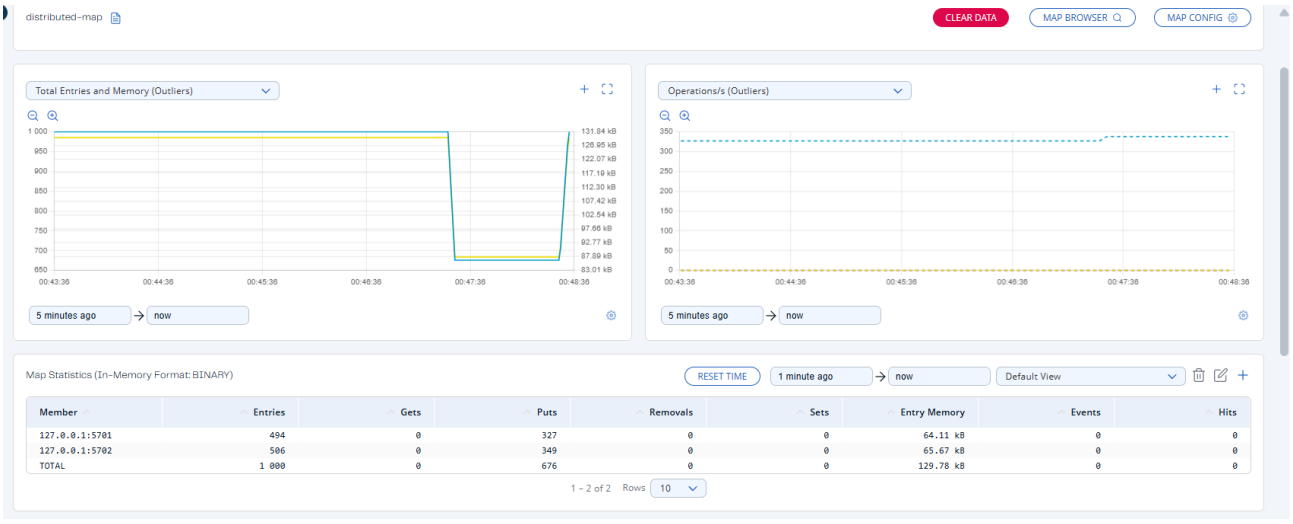
Hazelcast management system:



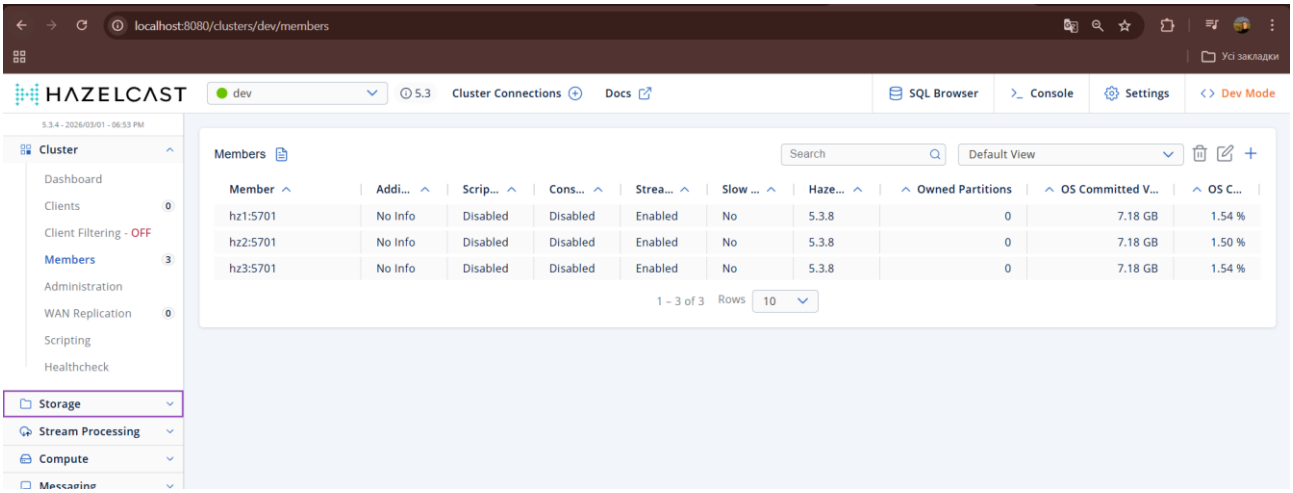
Distribution of map after 1000 uploading:



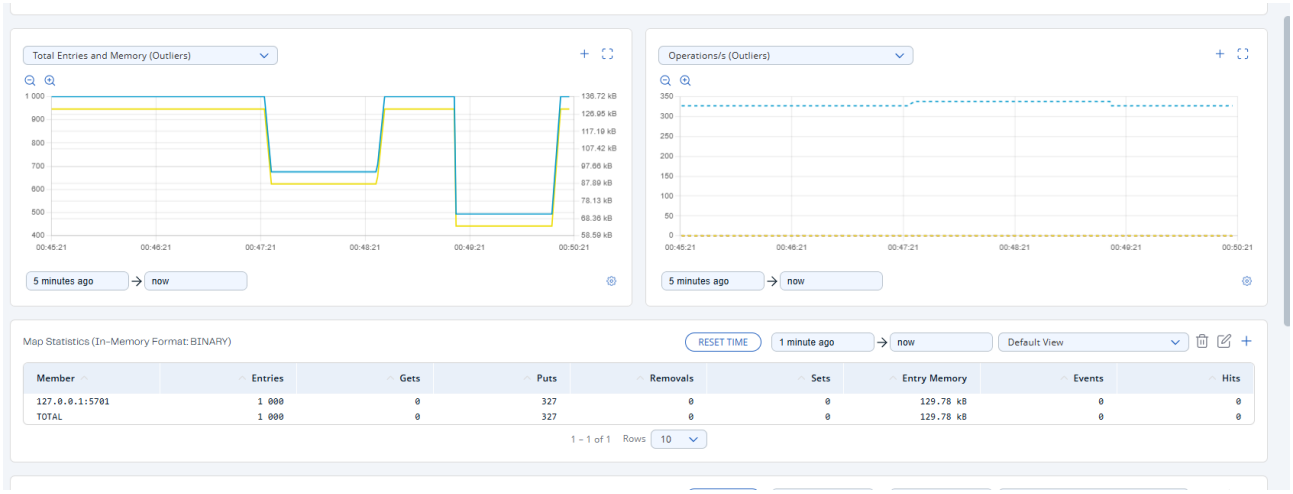
I have disabled 1 node:

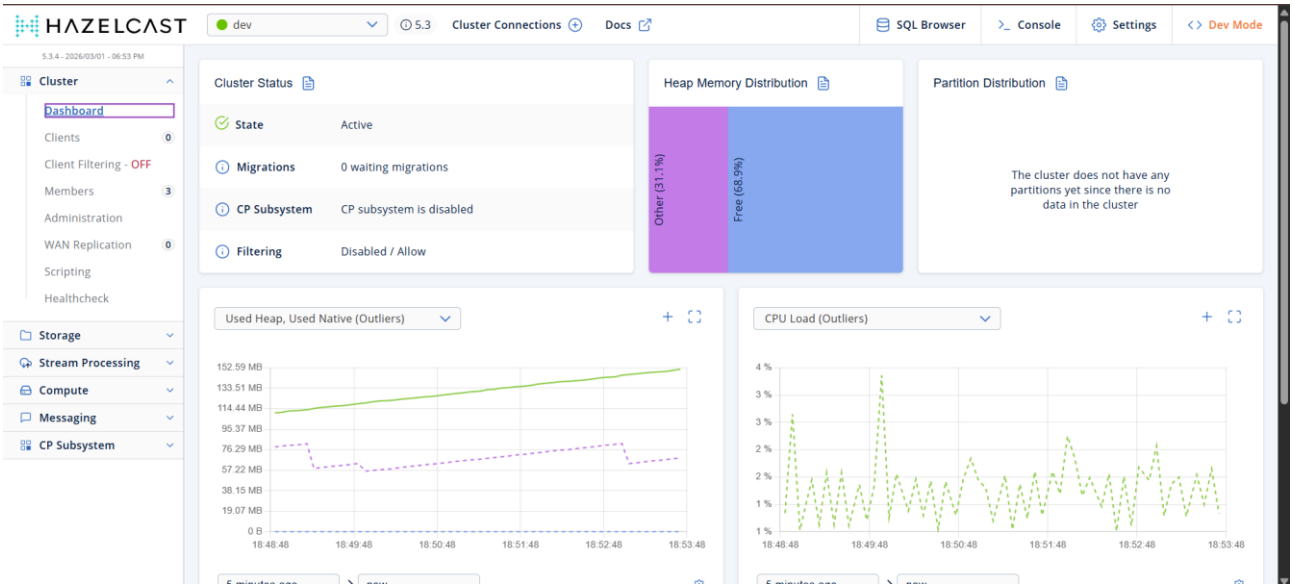


I have disabled the second node

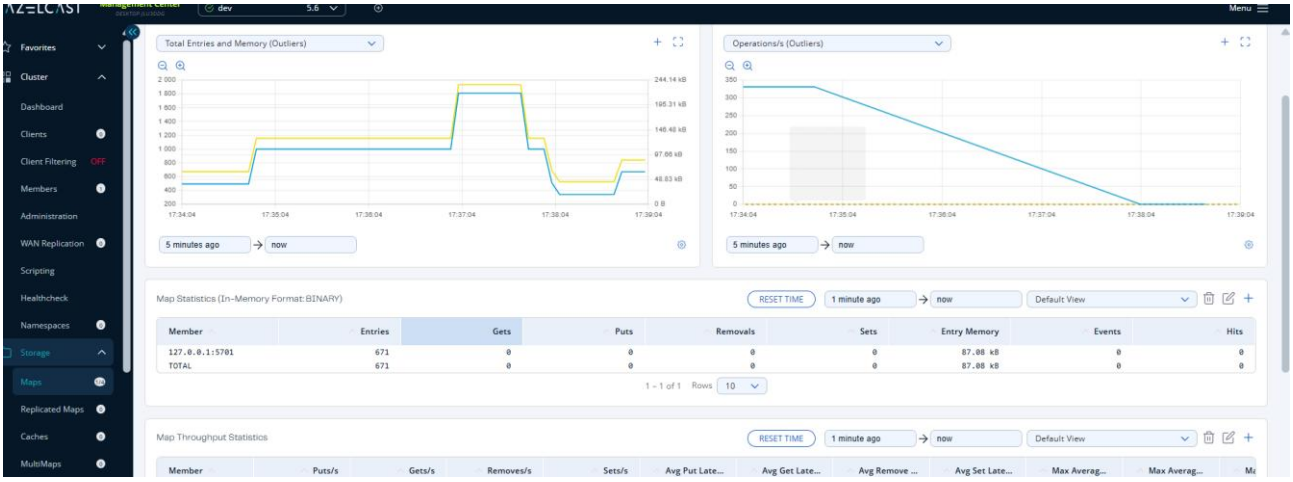


3) I have disabled the 2nd node:





Now I have disabled 2 simultaneously. And I see the losing of data (671 < 1000 entries):

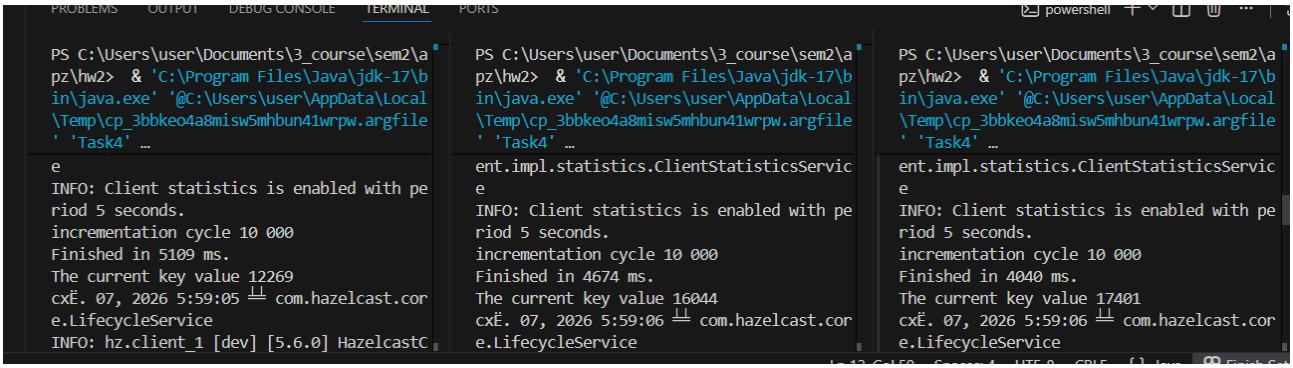


The task 4:

I have run the Task4.java

The experiment shows a final key value significantly lower than 30,000 because the code lacks locking mechanisms, leading to a classic race condition where concurrent updates are lost. Although the Management Center correctly records approximately 30,003 get and put operations—representing 10,000 iterations from each of the three clients plus initialization and final checks—many of these operations were redundant. Because multiple clients frequently read the same "old" value at the same time, they incremented it locally to the same "new" number and performed separate put requests that simply overwrote each other. So, finally I got the current value of 17401. This confirms that while the cluster faithfully processed every incoming network request, the lack of atomic synchronization caused the final data state to be inconsistent.

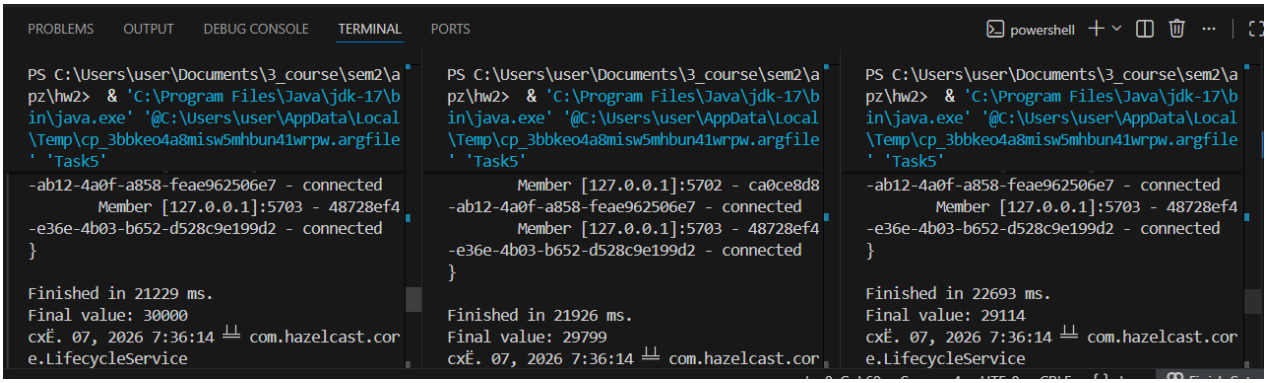
3 terminals:

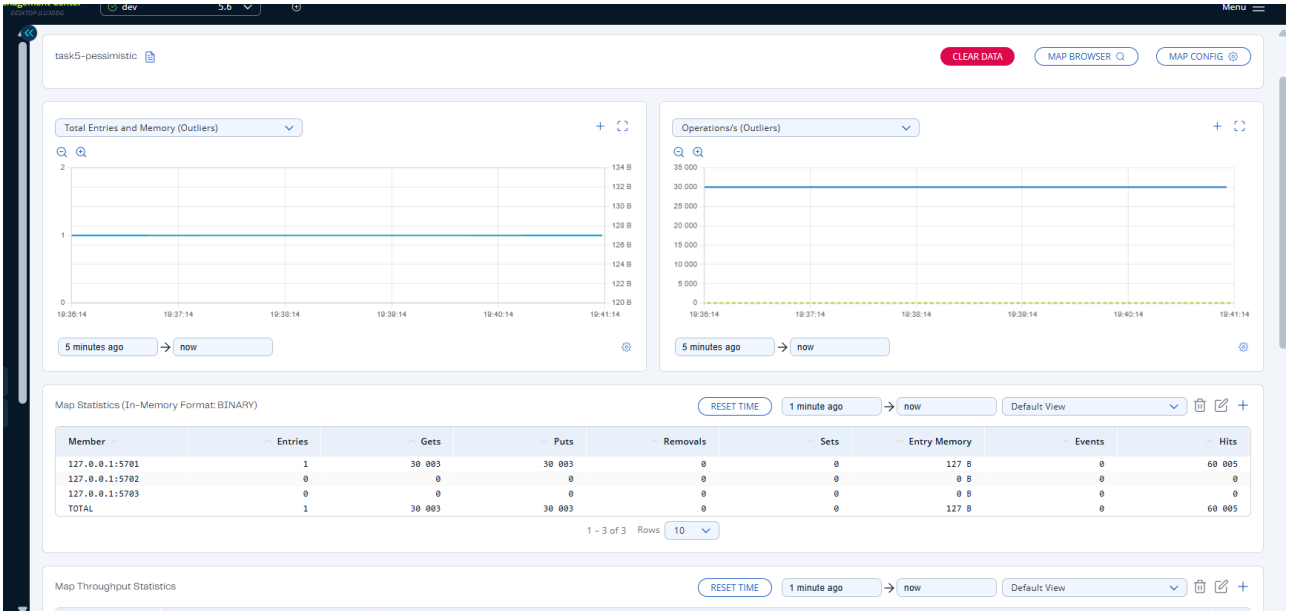


Task 5:

3 terminals:

I ran the pessimistic lockingexperiment across three terminals and successfully reached the target value of 30,000, confirming that the race condition from the previous task was fully resolved. By following the documentation to implement map.lock(key) and map.unlock(key), I forced the clients t wait for one another, ensuring every single increment was recordd correctly without any updates being lost. While my terminals finished with different intermediate valus like 29,799 and 29,114, these simply represented the state of the cunter at the exact moment those specific clients finished their loops while the others were still committing their final updates. I noticed that the execution time increased significantly to aproximately 22,000 ms, which highlights the performance trade-off required to achieve absolute consistency in a distributed environment



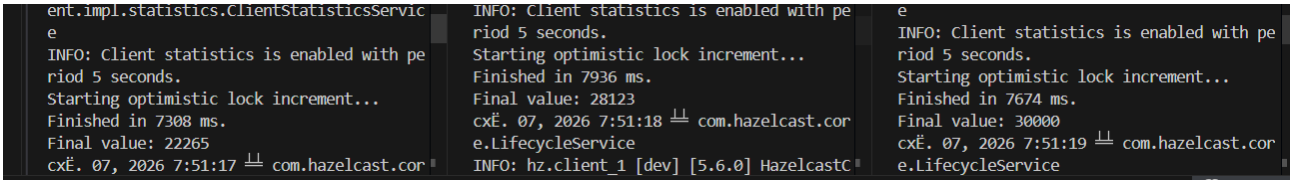


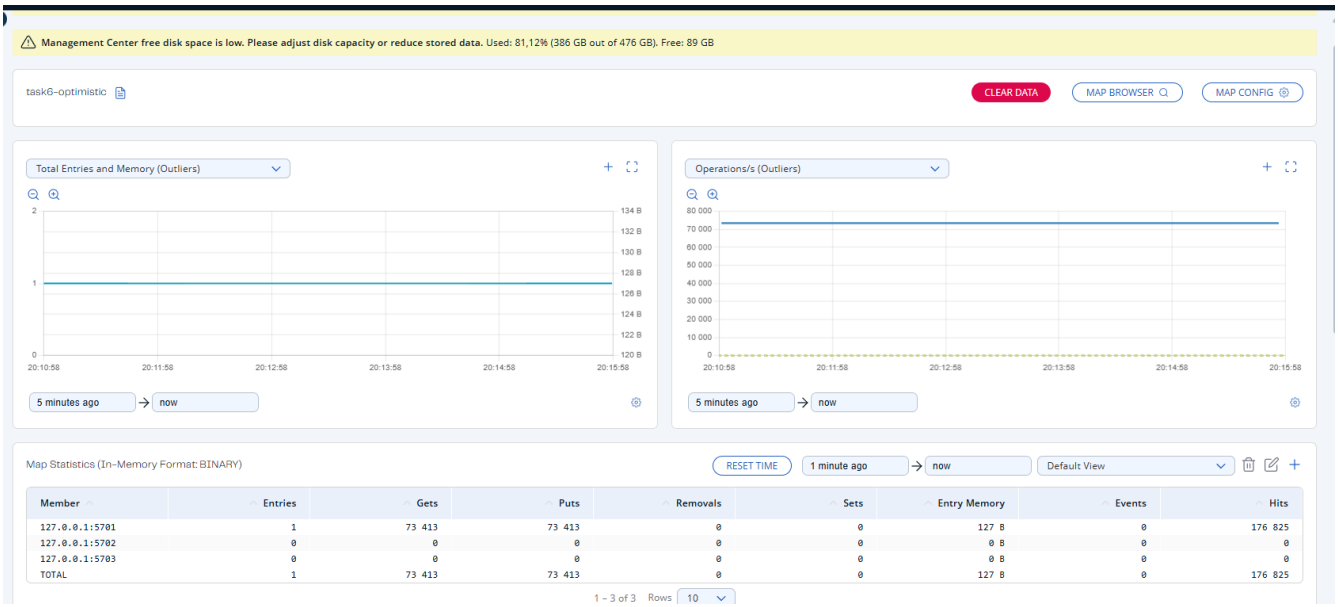
Task 6:

I completed the Optimistic Locking task using three concurrent terminals, achieving the target value of 30,000 with a significant performance improvement.

By using the map.replace(key, oldValue, newValue) method, the clients finished their loops in approximately 7,300–7,900 ms. This is nearly three times faster than the pessimistic approach, fulfilling the "performance boost" promised in the documentation for low-lock environments. While each terminal finished at different times, the final counter correctly reached the cumulative total of 30,000.

The Management Center statistics reveal that this speed comes from high retry traffic. Because the clients were competing for the same key, many replace calls failed when the oldValue had already been changed, forcing a retry. This resulted in 73,413 Gets and Puts—more than double the original operation count





Task 7:

Data I ntegrity Comparison

- No Locking (Task 4): This implementation resulted in significant data loss, yielding a final value of only 17,401 instead of the expected 30,000. This confirms that without atomic synchronization, concurrent updates overwrite each other in a classic race condition.
- Pessmistic and Optimistic Locking (Tasks 5, 6): Both implementations sucessfully reached the target value of 30,000. These methods ensured that every single increment was recorded correctly, resolving the consitency issues found in the non-locked version.

Performance Comparison: Pessimistic vs. Optimistic

- Pesimistic Locking: This was the slowest approach, taking approximately 22000 ms to complete. The high execution time is due to the performance tradeoff of forcing clients to wait in a queue for a networkbased lock.
- Optimistic Locking: This approach was significantly faster, finishing in aproximately 7,300–7,900 ms. This is nearly three times faster than the pessimistic approach.

Task 8:

1. I have created a new config file for the Queue.

2. I changed the original config file to my custom file (your path can be different from my ones):
   Copy-Item -Path "C:\Users\user\Documents\3_course\sem2\apz\hw2\hazelcast.xml" -Destination "C:\Users\user\Documents\3_course\sem2\apz\hw2_hazelcast_sets\hazelcast-5.6.0\config\hazelcast.xml" -Force

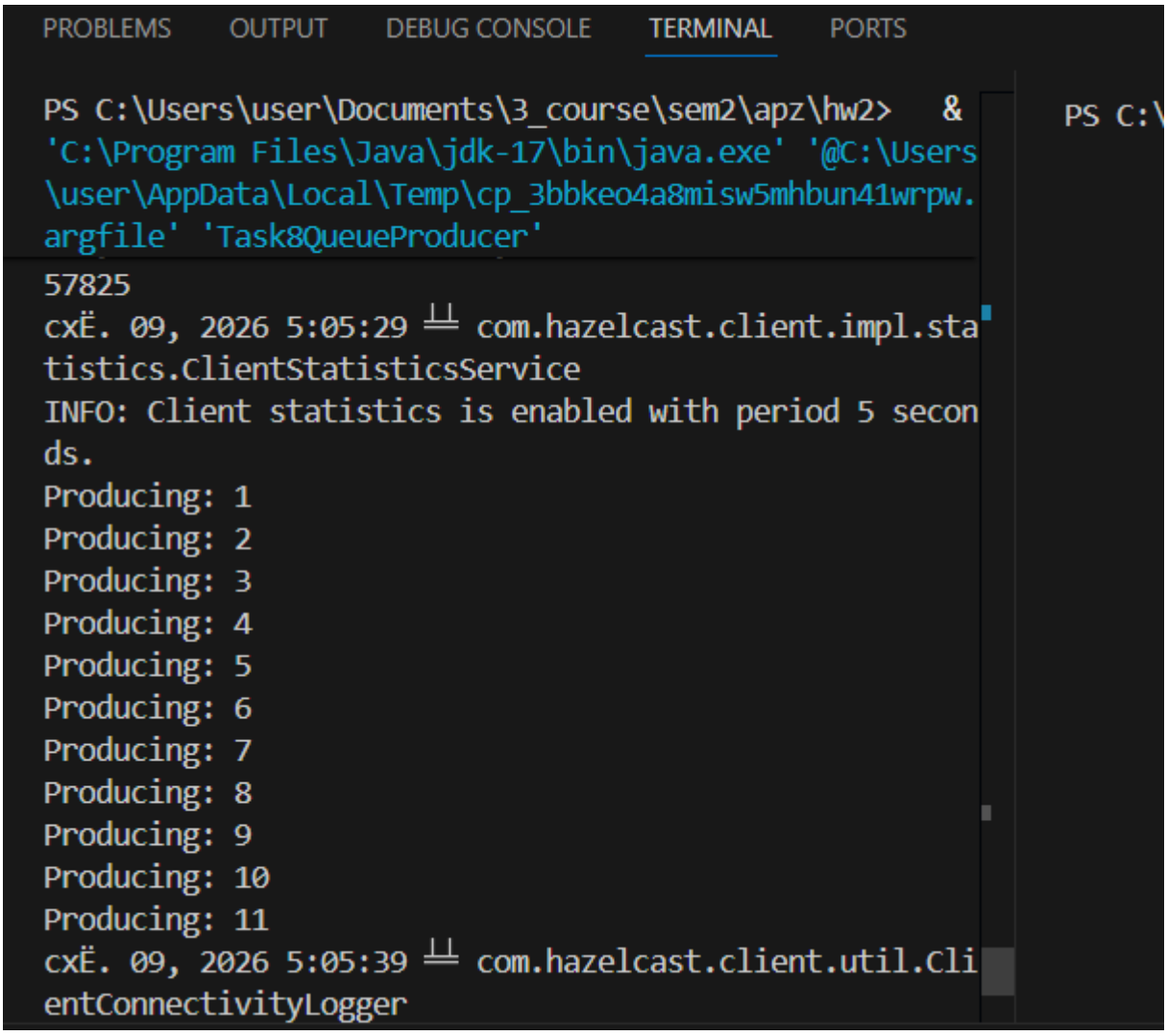
3. I cleaned environmental variables:
   $env:HAZELCAST_CONFIG=""
   $env:JAVA_OPTS=""

4. Restarted all the nodes:

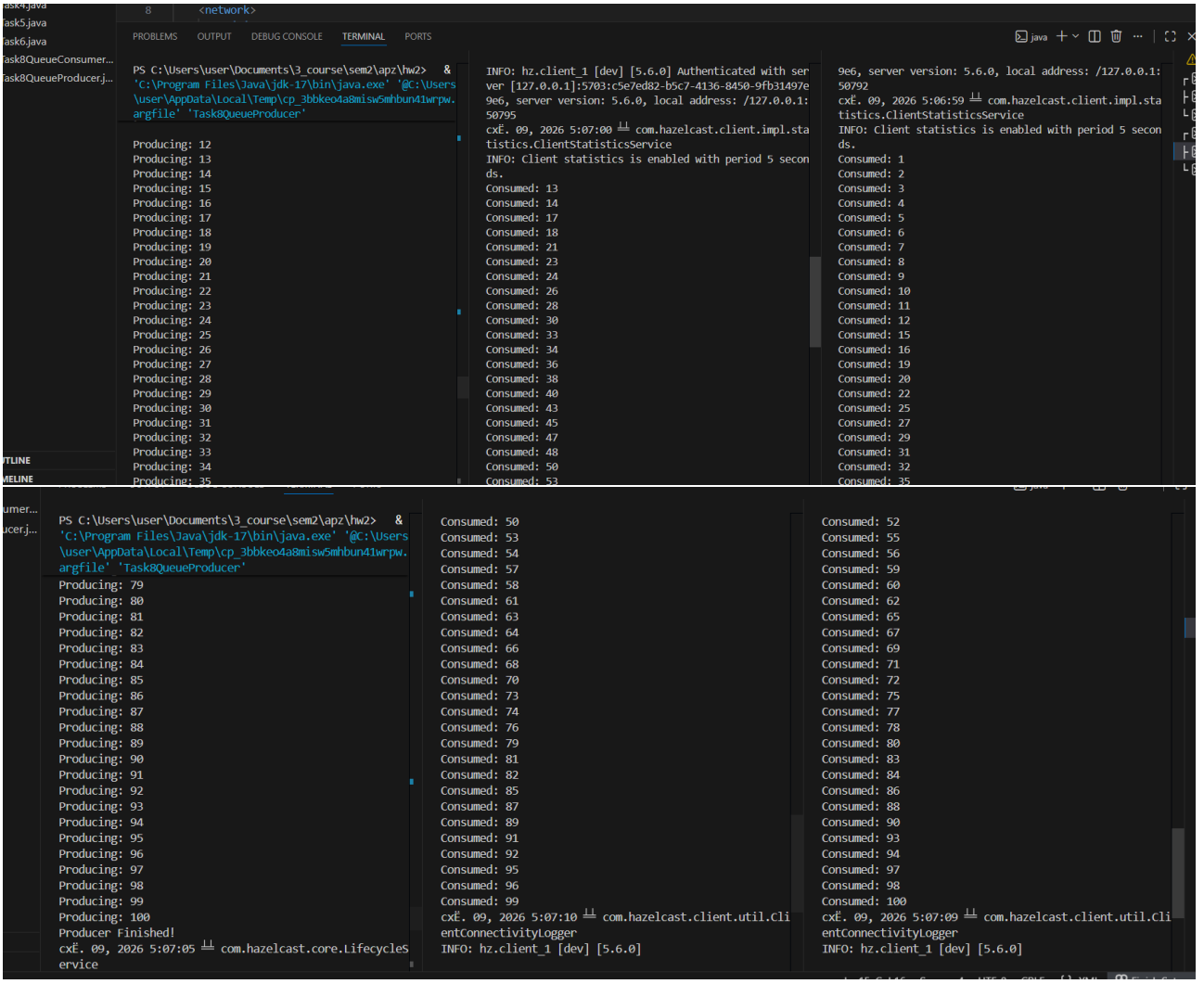
cd "C:\Users\user\Documents\3_course\sem2\apz\hw2_hazelcast_sets\hazelcast-5.6.0\bin\"

./hz-start.bat

5. I run the Queue Producer:



6. Next I run 2 Queue Consumers. When consumers started to consume:



2. Producer Behavir (Queue Full)

I ran a single Producer to write values 1 through 100.

1) The Producer printed Producing: 1 through Producing: 11 and then blocked (stalled).

2) Analysis: Because the queue limit was 10, the 11th put() operation was forced to wait. This cnfirms that when the queue is full and no consumers are present, the producer thread is suspended until space becomes available.

3) I started two concurrent Consumer clients to drain the queue.
4) As soon as the consumers started, the Producer resumed. The two consumers split the workload (e.g., Consumer 2 took items 1-12, while Consumer 1 took 13-14).

Reading Pattern: The values were read using the Competing Consumers pattern. Each message was processed imediately and only by one of the two clients. No data was duplicated or lost.

There is zero overlap between thtwo terminals. For example, while Consumer 1 (left) processed items 50, 53, and 54, Consumer 2 (right) was simultaneously processing items 52, 55, and 56. Once an item is takenby one client, it is instantly removed from the distriuted queue, making it unavailable to the other.

How are values read by two clients?

They are distributed on a firstcome, firstserved basis. Hazelcast ensures that while multiple clients can listen to the same queue, each individual item is delivered to only one consumer, effectively balancing the procesing load.

Write behavior when full?

The IQueu.put() method is a blocking operation. It prevents the producer from overfilling the queue, providing natural backpressure in the system.